Lecture 05.03: Attributes & Methods

📌 Executive Summary

Dot notation, internal **dict** mapping, method binding, dynamic attribute creation, and hasattr/getattr built-ins.

🔬 Theoretical Foundations & Principles

1. Attribute Storage via `__dict__`

In standard CPython classes, instance attributes are stored in a per-instance dictionary called `__dict__`:

$$\text{obj}.x = v \iff \text{obj.__dict__}['x'] = v$$

2. Method Binding & Callables

When an instance accesses a function defined in a class, Python transforms the function into a **bound method** using the descriptor protocol, automatically passing `self` as the first argument:

$$\text{obj.method}(a) \equiv \text{Class.method}(\text{obj}, a)$$

💻 Production Implementation & Code

```
class Product:
    def set_details(self, name: str, price: float):
        self.name = name
        self.price = price

p = Product()
p.set_details("Laptop", 1200.0)

print("Instance dictionary:", p.__dict__)
```

```
print("Has price?", hasattr(p, "price"))
print("Get price safely:", getattr(p, "price", 0.0))
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.